Instituto Superior Técnico

Parallel and Distributed Computing

Project Assignment

DOCUMENT CLASSIFICATION

Version 1.1 (19/02/2026)

2025/2026

3rd Period

Contents

1 Introduction **2**

2 Problem Description **2**

3 Implementation Details **2**
3.1 Input Data 2
3.2 Output Data 3
3.3 Computing Distances 3
3.4 Document Distribution 3
3.5 Sample Problem 3
3.6 Measuring Execution Time 4

4 Part 1 - Serial implementation **4**

5 Part 2 - OpenMP implementation **5**

6 Part 3 - MPI implementation **5**

7 What to Turn in, and When **5**
7.1 Deadlines 6

Revisions

Version 1.0 (February 12, 2026)	Initial Version
Version 1.1 (February 19, 2026)	Removed mandatory use of sqrt.

1 Introduction

The purpose of this class project is to give students hands-on experience in parallel programming on both shared- and distributed-memory systems, using OpenMP and MPI, respectively. For this assignment you are to write a sequential and two parallel implementations of a Document Classification algorithm.

2 Problem Description

Given a set of D documents, with each document classified according to S subjects, and a number C of cabinets, assign documents to cabinets according to the similarity of subjects.

The classification of a document is obtained by giving it a score to each subject, thus obtaining for each document a vector with S dimensions.

The decision of which document goes into which cabinet is made by trying to minimize the overall "distances" of the subjects of the document assigned to a cabinet (maximize subject similarity).

We will be using the following approximate algorithm to solve this problem:

1. make an initial assignment of the documents in D to the C cabinets;
2. for each cabinet, compute the "average" for each subject of its documents;
3. for each document, compute the distance to the averages of each cabinet and move the document to the cabinet with shorter distance;
4. go to Step 2 until no documents switch cabinets.

3 Implementation Details

3.1 Input Data

The description of the problem is contained in a text file with the following format:

- first line with three integers indicating, in order:
  - number of cabinets, C
  - number of documents, D
  - number of subjects, S
- after there will be D lines, each line corresponding to a document and with the following information:
  - id of the document, an integer in the range $[0, D[$
  - score of the document per subject, a set of S floating point values in order separated by a space

Your program should thus allow a single command-line parameter, the name of this file.

3.2 Output Data

Your program should simply send to the standard output (`stdout`) the number of the cabinet to which the document belongs to, a single integer per line ordered by document number.

The submitted programs should send these output lines (and **nothing else!**) to the standard output, so that it can be validated against the correct solution. The project **cannot be graded** unless you strictly follow these rules!

3.3 Computing Distances

Assume that we have two points a and b in a 2-dimensional space, with coordinates (a_x, a_y) and (b_x, b_y) respectively. The distance between these two points in a 2-dimensional space is given by the following expression:

$$\delta = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}.$$

In a S-dimensional space, the case of the Document Classification algorithm, to compute the distance between a document d and a cabinet c we can use the following expression:

$$\delta = \sqrt{\sum_{i=0}^{S-1} (d_i - c_i)^2},$$

where S is the number of subjects and d_i and c_i are the score of subject i of the document and of the cabinet.

The score of the cabinet is the average of the scores of the documents the cabinet holds at any given time.

Please use the type `double` to represent the floating point values to avoid numerical errors.

3.4 Document Distribution

In order to guarantee a deterministic result, the start cabinet assignment must be predefined. Thus, a round-robin assignment should be performed initially where each document is placed in the cabinet corresponding to the remainder of the document index i divided by the number of cabinets C, `i % C`.

There is no limit to the number of documents in a cabinet. In particular, there is no requirement about the balancing of the distribution of documents per cabinets. In the unlikely case a cabinet becomes empty, place its centroid at the origin (all coordinates at 0).

3.5 Sample Problem

Assuming that we have a problem with 4 cabinets, 10 documents and 3 subjects, we define this in a file `ex.in` with the following content:

```
4 10 3
0 1.2 1.0 10.3
1 2.8 1.4 1.0
2 6.0 2.6 1.1
3 5.1 2.3 2.9
4 8.35 1.6 4.3
5 2.0 2.7 2.2
6 6.4 1.8 8.4
```

```
7 5.2 4.7 1.4
8 1.1 1.7 4.3
9 7.1 3.1 2.0
```

For this problem the output should be:

```
$ ./docs ex.in
0
1
3
3
2
1
0
3
1
3
```

3.6 Measuring Execution Time

To make sure everyone uses the same measure for the execution time, the routine `omp_get_wtime()` from OpenMP will be used, which measures real time (also known as "wall-clock time"). This same routine should be used by all three versions of your project.

Also, we will not consider the time to read the input file.

Hence, your programs should have a structure similar to this:

```
#include <omp.h>
<...>

int main(int argc, char *argv[])
{
    double exec_time;

    <process input file>
    exec_time = -omp_get_wtime();
    <main program execution>
    exec_time += omp_get_wtime();
    fprintf(stderr, "%.1fs\n", exec_time);
    print_result();       // to the stdout!
 }
```

In this way, the execution time only accounts for the running time of the actual algorithm, and this value is sent to the standard error, `stderr`. The use of these separate output stream allows the validation of the results and analysis of execution time to be performed indepedently.

Because this time routine is part of OpenMP, you need the include `omp.h` and compile all your programs with the flag `-fopenmp`.

4 Part 1 - Serial implementation

Write a serial implementation of the algorithm in C or C++. Name the source file of this implementation `docs.c`.

Your program should accept a single command line argument, which will be the name of a file with the problem description.

This will be your base for comparisons and it is expected that it is as efficient as possible.

5 Part 2 - OpenMP implementation

Write an OpenMP implementation of the algorithm, with the same rules and input/output descriptions. Name this source code `docs-omp.c`.

You can start by simply adding OpenMP directives, but you are free, and encouraged, to modify the code in order to make the parallelization more effective and more scalable. Be careful about synchronization and load balancing!

Do not have hardcoded in your program the number of threads to use (unless for some reason your parallel strategy requires so) because we will be using the environment variable `OMP_NUM_THREADS` to evaluate the scalability of your program.

Create a table with the runtimes and speedups of this parallel version on input data that will be made available, for 1, 2, 4 and 6 parallel tasks (you may use Labs 2, 3, or 6 for this).

6 Part 3 - MPI implementation

Write an MPI implementation of the algorithm as for OpenMP, and address the same issues. Name this source code `docs-mpi.c`.

For MPI, you will need to modify your code substantially. Besides synchronization and load balancing, you will need to take into account the minimization of the impact of communication costs. You are encouraged to explore different approaches for the problem decomposition.

You may assume that all computacional nodes have access to the input file.

Create a table with the runtimes on Deucalion and speedups of this parallel version on input data that will be made available, for powers of 4 (1, 4, 16, ...). Note the CPU available on Deucalion is limited, you should test and tune your program on the lab computers before running on Deucalion.

You may also consider a combined MPI+OpenMP implementation.

7 What to Turn in, and When

You must eventually submit the sequential and both parallel versions of your program (**remember to use the filenames indicated above** – the executables should have the same name as the source without the extension).

You are **not to use** AI tools in the project, it will be considered as fraud, just like if you used someone else's code. All project submissions will be submitted to a system for detecting software similarity[1] and the use of such tools will be flagged by it.

For both the OpenMP and MPI versions (not for serial), you must also submit a short report about the results (2-4 pages) that discusses:

- the approach used for parallelization
- what decomposition was used
- what were the synchronization concerns and why

[1] `https://theory.stanford.edu/~aiken/moss`

- how was load balancing addressed
- the performance results, and if they are what you expected

The code, makefile and report will be uploaded to the Fenix system in a zip file. **Name these files** as `g<n>serial.zip`, `g<n>omp.zip` and `g<n>mpi.zip`, where `<n>` is your group number. The structure of this zip file should be:

```
g<n>/
├─ version/ (serial/omp/mpi)
│  ├─ src/
│  │  ├─ main file written in C/C++
│  │  ├─ Makefile
│  │  └─ other files required for compilation
│  └─ report in PDF (only OpenMP / MPI implementations)
├─ previous_version/ (if applicable)
└─ previous_version/ (if applicable)
```

that is:

- it should contain a single directory named `g<n>`, everything else is under this directory;
- this directory should have one to three directories, in the first submission just one named `serial`, in the second an additional directory `omp`, and in the final submission an extra `mpi` directory. The contents of the directories may change between submissions, but be sure to state that in the report;
- each of these directories will have two items:
  - your report in PDF;
  - a directory named `src` with your code and respective makefile.

Some final notes:

- the command "make" should generate your executable in the respective `src` directory, no flags/parameters should be needed. Also, the makefile should use the gnu compiler with the optimization flag `-O2`;
- the parallel implementations should work with the OpenMP and MPI versions installed in the lab computers;
- your submission will be evaluated automatically, and this will only work if you adhere to the naming convention and directory structure indicated.

7.1 Deadlines

1st due date, serial version: **March 4th**, until 23:59.

2nd due date (OpenMP): **March 18th**, until 23:59.

3rd due date (MPI): **April 3rd**, until 23:59.